

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO-4-ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	S.MOHAMED NAZEER	Reg. No.:	19424084
Section / Batch:	AI & DS	Date:	19/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Q22. Job Scheduling with Deadlines

Each job has a deadline and a profit, and only one job can be scheduled at a time. The objective is to schedule jobs such that total profit is maximized while ensuring each job is completed before its deadline. Conflicts must be avoided, and optimal ordering is required. Use a Greedy strategy to determine the best schedule.

Sample Input:

n = 4

deadline = 2 1 2 1

profit = 100 19 27 25

Expected Output:

Max Profit = 127

AIM

To develop a Python program using the Greedy strategy to schedule jobs with deadlines and profits so that the total profit is maximized while completing each selected job before its deadline.

ALGORITHM

1. Read the number of jobs, deadlines, and profits.
2. Create a list containing each job with its deadline and profit.
3. Sort all jobs in descending order of profit.
4. Find the maximum deadline and create available time slots.
5. For each job, assign it to the latest free slot before or on its deadline.
6. If a suitable slot is available, schedule the job and add its profit to the total profit.
7. Display the selected job order and maximum profit.

OBJECTIVE

1. To understand and implement the **Job Scheduling with Deadlines** problem using the **Greedy Algorithm**.
2. To analyze each job based on its **deadline and associated profit** and determine the most profitable scheduling order.
3. To arrange the jobs in **descending order of profit** so that higher-profit jobs are given priority during scheduling.
4. To assign each selected job to the **latest available time slot before its deadline**, ensuring that all deadline constraints are satisfied.
5. To avoid scheduling conflicts by ensuring that **only one job is assigned to each available time slot**.
6. To calculate and maximize the **total profit** obtained from the successfully scheduled jobs.
7. To demonstrate how a Greedy strategy can provide an **optimal job sequence** for maximizing profit under deadline constraints.
8. To implement the scheduling process using **Python** and visualize the selection and scheduling steps clearly.

PSEUDO CODE:

BEGIN

Read N

FOR i = 1 to N

Read Job[i], Deadline[i], Profit[i]

END FOR

Sort all jobs in DESCENDING order of Profit

MaxDeadline $\leftarrow$ maximum deadline among all jobs

Create Slot[1...MaxDeadline]

Mark every Slot as EMPTY

TotalProfit $\leftarrow$ 0

FOR each Job in sorted order

Deadline $\leftarrow$ Job.Deadline

Profit $\leftarrow$ Job.Profit

FOR Slot $\leftarrow$ Deadline DOWNT0 1

IF Slot[Slot] is EMPTY THEN

Slot[Slot] $\leftarrow$ Job

TotalProfit $\leftarrow$ TotalProfit + Profit

PRINT "Job scheduled:", Job

PRINT "Time slot:", Slot

BREAK

ELSE

PRINT "Slot occupied, checking previous slot"

END IF

END FOR

END FOR

PRINT "Final Schedule"

FOR Slot $\leftarrow$ 1 to MaxDeadline

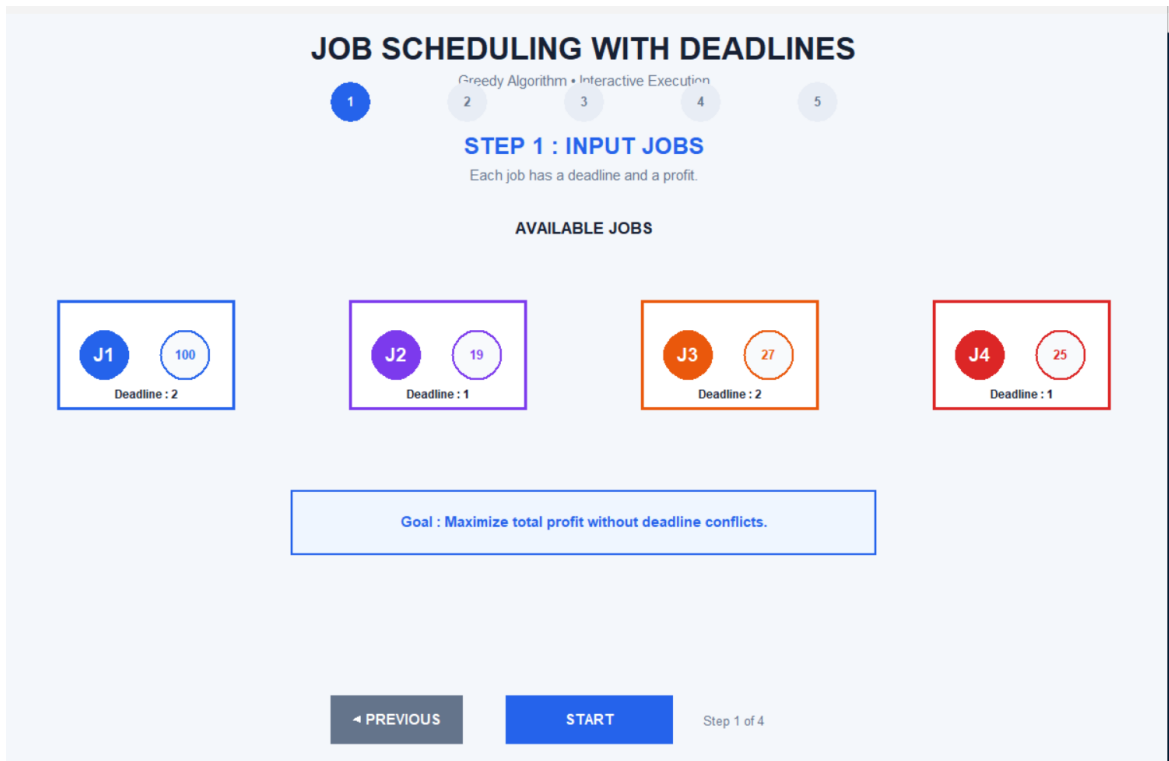
PRINT Slot, Slot[Slot]

END FOR

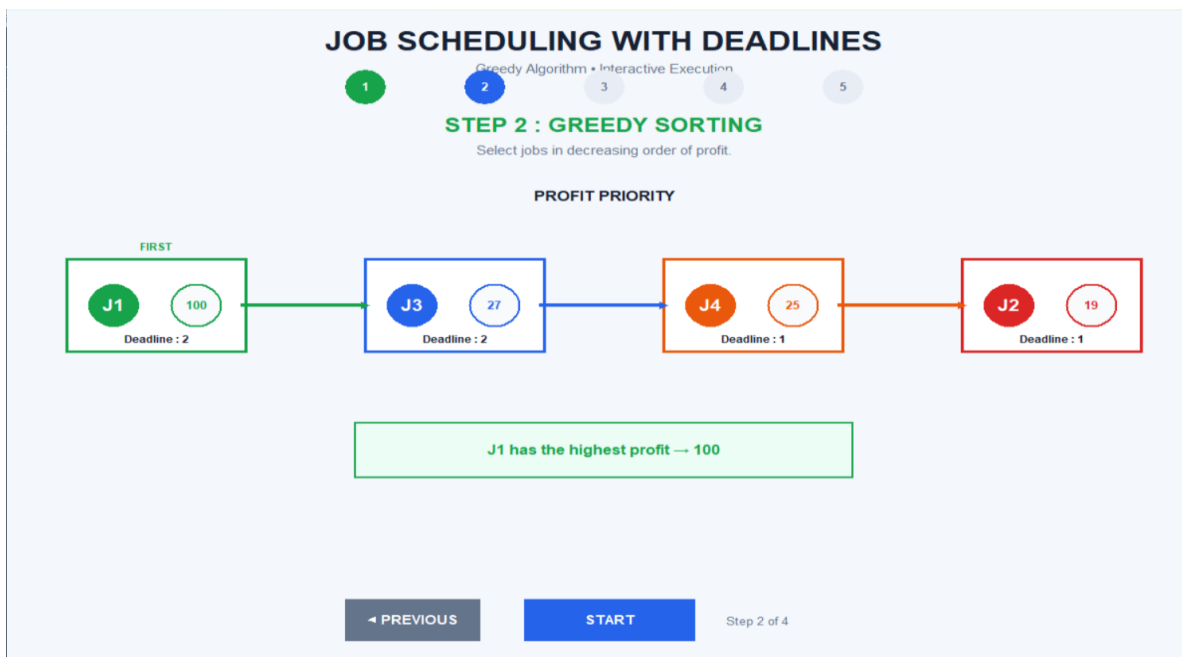
PRINT "Maximum Profit =", TotalProfit

END

INPUT:



PROCESS:



JOB SCHEDULING WITH DEADLINES

1

2

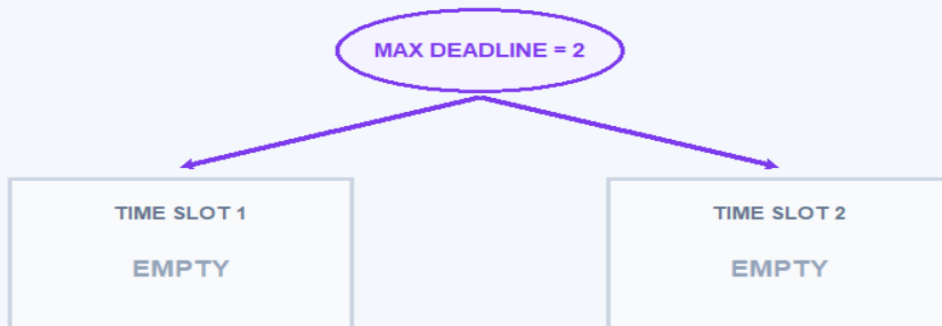
3

4

5

STEP 3 : CREATE TIME SLOTS

Maximum deadline is 2, so two slots are created.



Only one job can occupy each time slot.

◀ PREVIOUS

START

Step 3 of 4

JOB SCHEDULING WITH DEADLINES

1

2

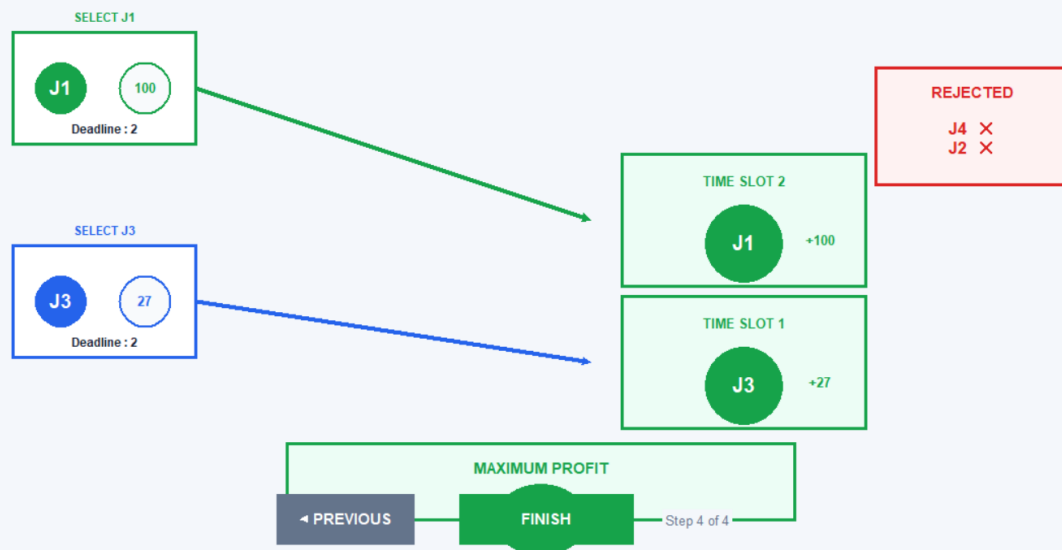
3

4

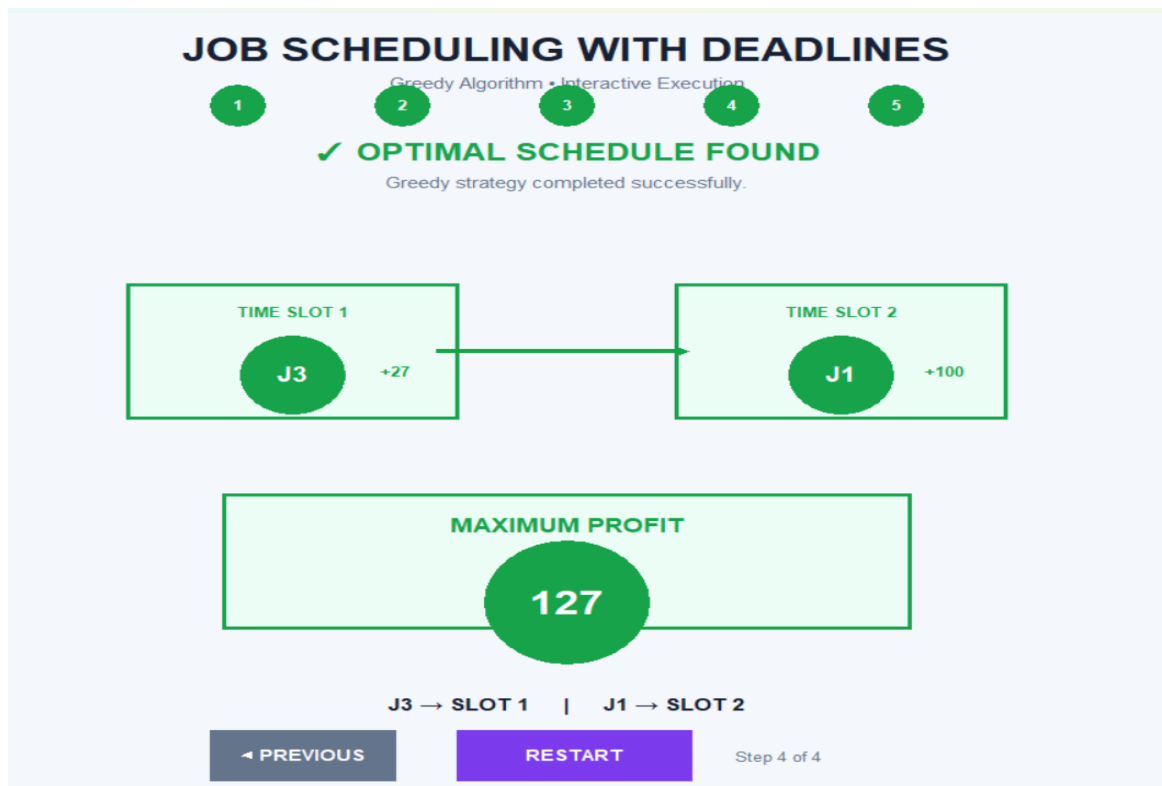
5

STEP 4 : GREEDY SCHEDULING

Place each job in the latest available slot.



FINAL OUTPUT:



RESULT AND CONCLUSION:

The Greedy algorithm successfully schedules the jobs **J3** and **J1** within their deadlines. The optimal schedule is **J3 → Slot 1** and **J1 → Slot 2**, giving a **maximum profit of 127**. Thus, the algorithm efficiently maximizes profit while satisfying all deadline constraints.

Code of Conduct

I SMOHAMED NAZEER (192424084) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **NAZEER**

RUBRICS FOR CALCULATION**Coding Challenge Task**

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				

Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____